

Quick Start Guide - Frontend Integration

What Was Built

We've built a complete frontend integration for the Mindous AI agent platform that allows users to:

1. **Chat with the AI Agent** - Send natural language requests to build apps
2. **Watch Real-Time Progress** - See live action streams as the agent works
3. **View Generated Code** - Syntax-highlighted code displays with copy functionality
4. **Monitor Builds** - Real-time build progress with logs
5. **Test Apps Instantly** - Live preview iframes with responsive controls

Components Created

Core Agent Components (/components/agent/)

1. **agent-message.tsx** - Main component that displays agent execution state
2. **action-stream.tsx** - Real-time action feed
3. **code-display.tsx** - Syntax-highlighted code viewer
4. **build-progress.tsx** - Build progress tracker with logs
5. **preview-frame.tsx** - Live preview iframe with viewport controls
6. **index.ts** - Export file for easy imports

Hooks (/lib/hooks/)

1. **useAgentStream.ts** - SSE connection hook for real-time updates

Updated Components

1. **chat-interface.tsx** - Integrated with agent execution
2. **message-list.tsx** - Added support for agent messages

File Structure

```
mindous/
├── components/
│   ├── agent/
│   │   ├── action-stream.tsx ✓ NEW
│   │   ├── agent-message.tsx ✓ NEW
│   │   ├── build-progress.tsx ✓ NEW
│   │   ├── code-display.tsx ✓ NEW
│   │   ├── preview-frame.tsx ✓ NEW
│   │   └── index.ts ✓ NEW
│   ├── chat/
│   │   ├── chat-interface.tsx ✓ UPDATED
│   │   └── message-list.tsx ✓ UPDATED
│   └── runs/
│       └── run-progress-panel.tsx ✓ UPDATED
├── lib/
│   └── hooks/
│       └── useAgentStream.ts ✓ NEW
└── docs/
    ├── FRONTEND_INTEGRATION.md ✓ NEW
    └── QUICK_START.md ✓ NEW (this file)
```

How It Works

1. User Sends Message

```
// User types: "Create a todo app"
handleSendMessage("Create a todo app")
```

2. Start Agent Execution

```
// Call agent execution API
const response = await fetch('/api/agent/execute', {
  method: 'POST',
  body: JSON.stringify({
    prompt: "Create a todo app",
    context: { taskType: 'code', complexity: 'medium' }
  })
});

const { executionId, runId } = await response.json();
```

3. Subscribe to Real-Time Stream

```
// useAgentStream hook connects to SSE endpoint
const { state, isConnected } = useAgentStream({
  runId,
  executionId,
  onAction: (action) => {
    // Action stream updates in real-time
  },
  onCodeGenerated: (artifact) => {
    // Code artifacts added to display
  },
  onBuildUpdate: (buildInfo) => {
    // Build progress updates
  },
  onPreviewReady: (url) => {
    // Preview iframe shows live app
  }
});
```

4. Render Agent Message

```
<AgentMessage
  content="Building your todo app..."
  agentState={{
    status: 'executing',
    progress: 45,
    actions: [...],
    codeArtifacts: [...],
    buildInfo: {...},
    previewUrl: 'http://localhost:3000'
  }}
/>
```

Testing the Integration

Start the Application

```
cd /home/ubuntu/mindous
npm install
npm run dev
```

Test Flow

1. **Open the chat interface** at `http://localhost:3000`
2. **Send a test message:**
 Create a simple counter app with Next.js
3. **Watch the magic happen:**
 - ☒ User message appears
 - ☒ Agent message shows "Planning execution..."
 - ☒ Action stream updates with each step
 - ☒ Code artifacts display when generated

-  Build progress shows real-time logs
-  Preview iframe loads the built app

Expected Timeline

1. **0s** - Message sent, execution starts
2. **2-5s** - Planning phase, subtasks created
3. **5-15s** - Code generation, artifacts displayed
4. **15-30s** - Building phase, progress shown
5. **30-45s** - Preview ready, iframe loads
6. **45s+** - Execution complete, all interactive

Key Features

1. Real-Time Action Stream

-  Task started: Planning execution
-  Code generated: app.tsx
-  Build started
-  Build completed
-  Preview ready

2. Code Display

- Line numbers
- Syntax highlighting
- Copy to clipboard
- Collapsible
- Multi-language support

3. Build Progress

- Visual progress bar
- Current step indicator
- Expandable logs
- Build duration
- Error handling

4. Live Preview

- Responsive viewport (desktop/tablet/mobile)
- Refresh button
- Open in new tab
- Loading states
- Error handling

Customization

Change Status Colors

Edit `agent-message.tsx` :

```
agentState.status === 'planning' && 'bg-blue-500/10 text-blue-500'
// Change to your preferred color scheme
```

Modify Viewport Sizes

Edit `preview-frame.tsx`:

```
const viewportSizes = {
  desktop: { width: '100%', height: '600px' },
  tablet: { width: '768px', height: '1024px' },
  mobile: { width: '375px', height: '667px' },
  // Add custom sizes
};
```

Add New Event Types

Edit `useAgentStream.ts`:

```
switch (event.eventType) {
  case 'YOUR_NEW_EVENT':
    // Handle your event
    break;
}
```

Common Issues & Solutions

Issue: SSE Not Connecting

Solution:

```
# Check if Redis is running
redis-cli ping

# Check run exists
curl http://localhost:3000/api/agent/run/{runId}
```

Issue: Preview Not Loading

Solution:

```
// Check preview URL
console.log('Preview URL:', agentState.previewUrl);

// Verify iframe sandbox permissions
sandbox="allow-scripts allow-same-origin allow-forms"
```

Issue: Code Not Displaying

Solution:

```
// Check if code artifacts exist
console.log('Artifacts:', agentState.codeArtifacts);

// Verify event is firing
onCodeGenerated: (artifact) => {
  console.log('Code generated:', artifact);
}
```

Next Steps

1. Enhance Syntax Highlighting

- Integrate Prism.js or highlight.js
- Add more language support

2. Add Code Editing

- Allow inline code modifications
- Request changes via agent

3. Improve Build Logs

- Add log filtering
- Syntax highlighting for logs
- Search functionality

4. Advanced Preview Controls

- Device emulation
- Network throttling
- Console output

5. Analytics

- Track execution times
- Monitor success rates
- User behavior analytics

Support

For issues or questions:

1. Check the [Frontend Integration Docs](#) (./FRONTEND_INTEGRATION.md)
2. Review the [Backend Documentation](#) (./BACKEND_ARCHITECTURE.md)
3. Inspect browser console for errors
4. Check React DevTools for state

Resources

- **API Documentation:** `/docs/API.md`
 - **Component Library:** shadcn/ui
 - **Styling:** Tailwind CSS
 - **State Management:** React Hooks
 - **Real-Time:** Server-Sent Events (SSE)
-

